AI Engineer Assessment

Hi,

Thank you for your interest in the AI Engineer position at Konecto. This technical assessment is designed to give us insight into your skills in agentic design, tool creation, handling of tabular data and architectural thinking — and into **how you work as an AI engineer**.

We assume you will use AI to write most or all of the code. That is expected and encouraged. **What we evaluate is whether you understand what was built and the decisions made underneath it.** In a follow-up interview we will ask you to walk us through your architecture and defend specific trade-offs.

Scenario: Series 76 Technical Assistant

1. Objective

A plant engineering team needs to choose **Series 76 Electric Actuators** and look up their technical data. Today they do it by searching the manufacturer's tables by hand. Your task is to build a complete technical assistant for them:

- A **backend** in Python with an AI agent that answers questions about a specific actuator and recommends models based on the user's requirements.
- A **native mobile app** where the user talks to the agent and resumes previous conversations.

The agent's core capabilities must include:

- **Specific Data Retrieval:** Accurately answering questions about a specific actuator when a user provides its unique Base Part Number.

- **Product Recommendation:** Recommending one or more suitable actuators based on a set of technical requirements provided by the user.

2. Provided Resources

- **Documents:**
  - `series_76_electric_data.pdf` — electrical data tables for a subset of the Series 76 actuators, split by enclosure, power supply and service type. **This is the only source of truth for your agent's numbers.**
  - `series_76_summary.pdf` — a document providing business context about the actuators to help you understand the data.
- **OpenRouter API Key:**
  - You may use any models available on OpenRouter; choose the ones that best fit each task. Explain your choices in the README, and make the models configurable through environment variables.
  - **Important:** The key has a hard spending limit of **USD 10**. When it is reached, the key stops working and it will not be topped up. Please use it efficiently.

Hint: To experiment without spending your key's budget (e.g., trying out how to extract the tables from the PDF), you may find Google AI Studio helpful: it offers a free API, with a low daily request limit.

3. Prerequisites

- A GitHub repository to host all the source code (backend and app).
- A local Python environment to develop and run the backend.
- Docker installed locally to build and run the containerized backend.
- Android Studio or Xcode, depending on the platform you choose for the app.

Part A — The Backend

4. Functional Requirements

- A REST service in **Python with FastAPI**.

- An agent orchestrated with **LangChain**, equipped with **at least one custom LangChain Tool** that queries the table data.
- An ingestion script (e.g., `ingest.py`) that processes `series_76_electric_data.pdf` and transforms it into the format used by your tool.
- **Multi-turn conversation:** the backend keeps the history per `session_id`. A follow-up such as "and how much power does it draw?" is resolved without repeating the part number.
- **Persisted history:** after restarting the backend, conversations can be resumed.
- **Edge cases:** invalid part numbers, missing data, ambiguous queries and requirements that no model meets are handled correctly.

API contract. Only these fields are fixed:

```
POST /api/conversation

Request:  { "session_id": "<conversation id>", "query": "<user's question>" }
Response: { "session_id": "<conversation id>", "answer": "<agent's answer>", ... }
```

`session_id` is optional in the request: without it, the backend starts a new conversation and returns its new `session_id` in the response. With it, the agent continues that conversation.

You design the rest of the response format. Part of the challenge is deciding what structure the app needs to display product data well and to build the debug screen.

Suggested endpoints. Besides the contract above, these are a **suggestion**; you may change their names and shapes if you explain why in the README.

```
GET  /api/sessions                conversations: id, title, date
GET  /api/sessions/{id}/messages  history to resume a conversation
POST /api/auth/login              optional, see section 7:
                                  { email, password } -> { token }
```

5. Technical Considerations

1. **LLM & Embeddings:** You must use the models available through the provided API key. Be mindful of the limited budget when selecting models for development and testing.

2. **Orchestration:** Use **LangChain** as the orchestration framework for the agent and its tools.
3. **Data Handling:** You are free to decide the best way to process and store the data from the tables for your tool to query. Your handling of structured tabular data is a key part of this evaluation.
   – The database must be **PostgreSQL**, running locally (e.g., with Docker Compose).
   – If you need semantic (vector) search, use the **pgvector** extension.
4. **Conversation storage:** conversations must survive a backend restart, so they cannot live only in memory.
5. **Backend:** The REST server must be implemented with **FastAPI**.
6. **Numbers:** The technical data the user sees must match the tables. The LLM must not invent or recall numbers.

Part B — The Mobile App

6. App Requirements

Native: **Kotlin with Jetpack Compose** or **Swift with SwiftUI**, your choice. The app has **three screens**:

1. **Conversations screen** (for end users): the list of saved conversations.
   – Open a previous conversation: its history loads and the agent continues with the same context.
   – Start a new conversation.
2. **Chat screen** (for end users): a multi-turn chat that keeps the `session_id`.
   – Product data is shown **clear and comparable**, not as a paragraph of text. How you achieve it is up to you.
   – It must not show tool names or other internal details.
3. **Debug screen** (for developers only): for each answer, the tools called, their parameters and the data they returned.
   – End users should not see it. How it is opened is up to you; document it in the README.

All screens handle **loading states and errors** (for example, when the backend does not respond).

The optional login adds a fourth screen (section 7).

Part C — Optional Features

7. Optional Features

These features are **not required**. Implement them only if you have time left after the required parts.

Login

Without login, the app works for a single user and lists all saved conversations. If you implement it, keep it simple:

- Login with email and password that returns a token. Passwords are stored hashed, never in plain text.
- A few pre-loaded test users, documented in the README.
- A login screen in the app, which then sends the token on every request (e.g., `Authorization: Bearer <token>`). The backend takes the user from the token.
- Each user sees and uses only their own conversations.
- Not required: a sign-up screen, password recovery, or login with Google/Apple (OAuth).

Voice messages

The user can send voice messages in addition to text. No separate speech key is provided: transcribe on the device, with an open-source model, or with the OpenRouter key (it counts toward the same budget). If you implement it, document in the README how it works and the decisions you made.

8. Deliverables

Submit a link to a single **Git repository**. It must contain:

- All **source code** for the backend and the app.

- A **script or pipeline** (e.g., `ingest.py`) that processes the raw PDF data and transforms it into the format used by your tool.
- The **processed data** (or clear instructions in the README on how to generate it).
- A **Dockerfile** to build the backend image and a **docker-compose.yml** that starts the backend and the database.
- A **.env.example** file with placeholder values, without real keys.
- A **README** that lets us run everything without asking you anything. It must include:
  - **Setup:** requirements and the environment variables to fill in.
  - **Backend:** how to build and run it with Docker, and how to access the API.
  - **Ingestion:** how to run it and where the processed data ends up.
  - **App:** how to build and run it on an emulator or a device, how to point it to the backend, and how to open the debug screen.
  - **Design and architecture:** a brief explanation of the main decisions and their trade-offs.
  - **Optional features**, if implemented: the test users for the login, and how voice messages work.
- Your AI assistant's **configuration directory** (e.g., `.claude/`, `.cursor/`), so we can see your real setup.

9. Submission

You have **one week** from the day you receive this assessment. Reply to the email in which you received it with a link to your GitHub repository.

Important — please ensure your submission meets the following:

- **Reproducibility:** Your repository must include complete, step-by-step instructions that allow us to replicate and test your solution. If we cannot reproduce it following your documentation, we won't be able to move forward in the evaluation process.
- **Robust Implementation:** Your solution must handle edge cases and unexpected inputs appropriately (e.g., invalid part numbers, ambiguous queries, missing data). The implementation should be general enough to work with scenarios beyond the provided examples, as it will be tested against various test cases of our own.